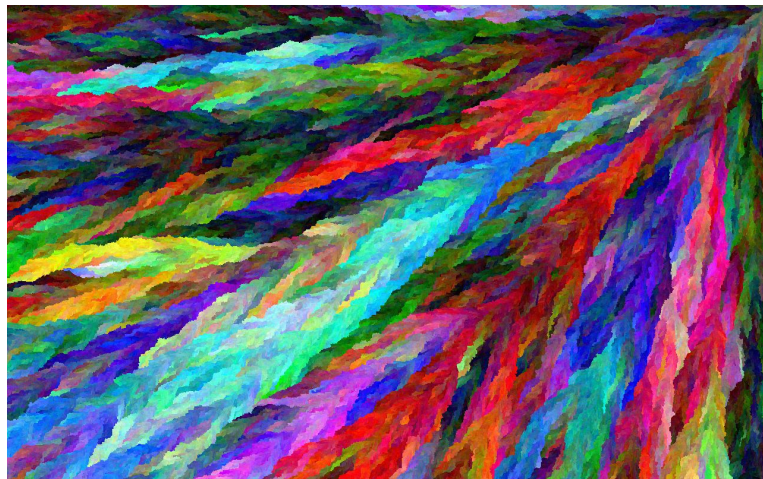


Course website: <https://fa25.datastructur.es>

Ask your questions in the Zoom chat! (link on website)



Lecture 1

Introduction to 61B, Java

CS61B, Fall 2025 @ UC Berkeley

Lecturers: Peyrin Kao, Josh Hug

Slides Credit: Josh Hug

Welcome to 61B

Lecture 1, CS61B, Fall 2025

- **Welcome!**
 - **Welcome to 61B**
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Layers of Abstraction

In 61A, you learned that programming is done in layers of abstraction.

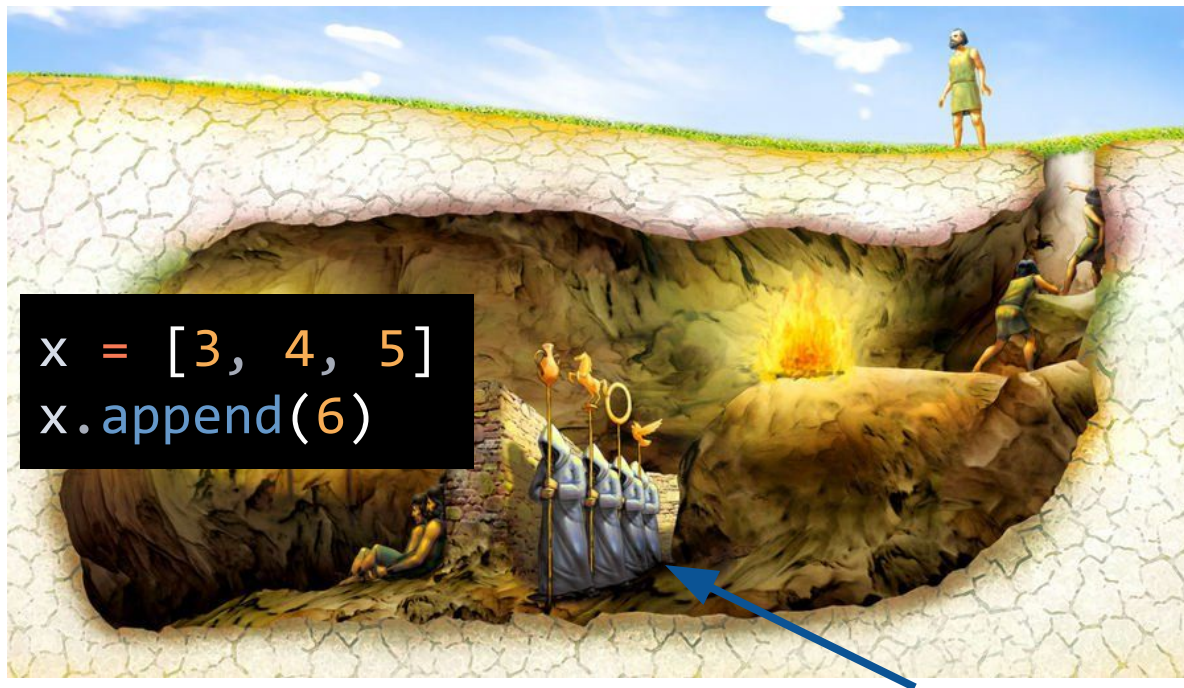
- You learned how to use a list.
- But never discussed precisely how a list was implemented.
 - (in case you're curious, here's the actual implementation:
<https://github.com/python/cpython/blob/main/Objects/listobject.c>)

```
x = [3, 4, 5]  
x.append(6)
```

Layers of Abstraction

In 61A, you learned that programming is done in layers of abstraction.

- You learned how to use a list.
- But never discussed precisely how a list was implemented.

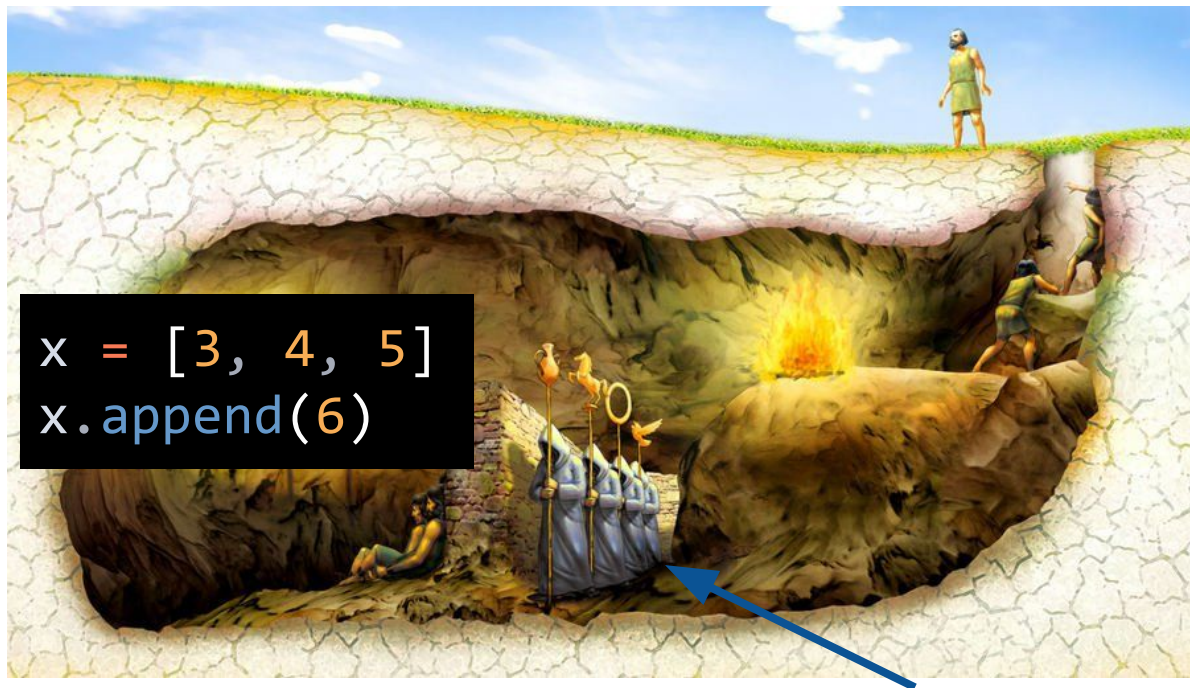


Us, in 61B

61B and Lists

Over the first four weeks of 61B, we'll explore two radically different choices for how one can implement a list.

- Along the way, we'll also learn more industrial strength techniques for building software.



Us, in 61B

What is 61B about?

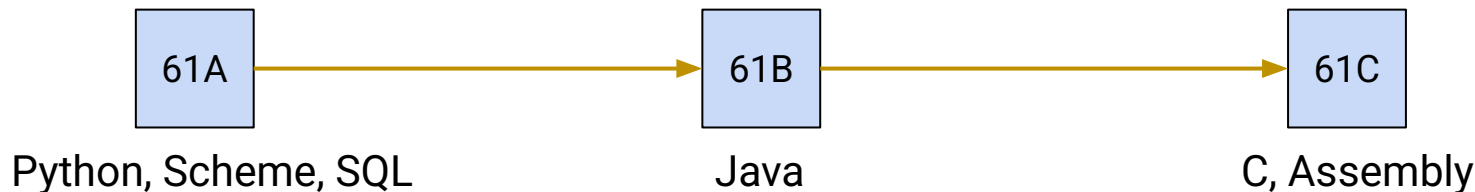
- Writing code that runs efficiently.
 - Good algorithms.
 - Good data structures.
- Writing code efficiently.
 - Designing, building, testing, and debugging large programs (including programs that implement and cleverly utilize data structures).
 - Use of programming tools.
 - git, LLMs, IntelliJ, JUnit / Truth, and various command line tools.

Assumes solid foundation in programming fundamentals, including:

- Object oriented programming, recursion, lists, and trees.

In our course, we'll use the Java programming language.

- Code runs much faster than Python.
- Includes interesting programming language features missing in Python.
 - Static typing.
 - Arrays.
 - Subtype polymorphism.
- Extremely popular language.



Other Reasons to Take 61B

Other great features of 61B:

- We cover the most popular topics for job interview questions in software engineering.
 - Examples: Hash tables, binary search trees, quick sort, graphs, Dijkstra's algorithm.
- Some really cool math. Examples:
 - Asymptotic analysis.
 - Resizing arrays.
 - The isometry between self-balancing 2-3 trees and self-balancing red black trees.
 - Graph theory.
 - P vs NP.
- Once you're done: the confident sense that you can build any software.

Josh Hug: Berkeley faculty since 2014.

- EE B.S. from UT Austin in 2003.
- EECS Ph.D. from UC Berkeley 2011.
- Taught at Princeton 2011-2014.
- Spent sabbatical two years ago in the **Netherlands**.
- **13th time** teaching this class, designed most of the homeworks and projects.
- Also teaching CS70 (!!).

Plug: **2025 Fall** [DUTCH 1 001 - LEC 001](#) offered through [German](#)

Elementary Dutch

👤 Esmee van der Hoeven

Aug 27, 2025 - Dec 12, 2025

📅 Mo, We

🕒 12:00 pm - 01:59 pm

📍 [Dwinelle B33B](#)

Aug 27, 2025 - Dec 12, 2025

📅 Fr

🕒 12:00 pm - 12:59 pm

Class #: 21352 Units: 5

Instruction Mode: In-Person Instruction

Open Seats

3 Unreserved Seats

This introduction to the Dutch language focuses on the development of basic communicative competencies (spoken and written). Students will be able to understand and use high frequency vocabulary and basic grammatical structures. This course is not open for native speakers of Dutch.



About Us (Peyrin)

Hi, I'm Peyrin.

Pronouns: he/him

<https://peyrin.github.io>

Question for You

What do you hope / expect to learn from this class? Why are you taking it?

- Need to take it for my major (physics) as a technical elective.
- Get a job.
- 61A was awesome.
- Make cool things.
- Data science major, fulfill requirements for major. Technical interviews, seems useful. Class seems interesting.
- I wanted to see professor josh hug.

Who are you?

- Freshman? Sophomore? Junior? Senior? Grad student? None of the above?
- CS Major? EECS? DS Major? Intending to be a CS/DS Major? Something else?
- CS 61A? Java experience? Experience with any other programming languages?

61B Logistics

Lecture 1, CS61B, Fall 2025

- **Welcome!**
 - Welcome to 61B
 - **61B Structure and Logistics**
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Joining the Course

Course staff does not control getting into the course.

- [Contact a major advisor](#) if you have questions.

Being added to course platforms (Ed, Gradescope) is **manual**. We do this every day. Please **do not** email us (cs61b@berkeley.edu) asking to be added to the platforms unless it's been more than 2 days since you joined the course.

If you are a pending concurrent enrollment student, you should be added to bCourses, Ed, and Gradescope **1–2 days after** you've submitted your application.

- We cannot process/approve applications until the department tells us to. We will post on Ed if there are updates.
- **Do not** email course staff about getting added to the course platforms unless it's been **more than 2 days** since you've submitted your application.

DSP (Disabled Students' Program)

If you have DSP accommodations, please submit your letter through the DSP portal as soon as possible!

Course Components

Lectures (~40/hours) provide you with an introduction and a foundation.

You'll learn most of what you learn in the class with **practice** (~160-200 hours):

- Programming (programming homeworks, projects, discussion section, old exams).
- Solving interesting problems (theory homeworks, old exams, discussion section).



This class is divided into three phases:

- Phase 1 (weeks 1 - 4): Intro to Java and Data Structures.
 - Learning core programming skills in the context of list implementations.
 - All coding work is solo.
 - Moves VERY fast. Lots of little deadlines.
 - Two mini-projects on data structures.
 - Midterm 1 will test programming skills developed on mini-projects.
- Phase 2 (weeks 5 - 10): Data Structures (and a little Software Engineering).
 - All coding work is solo.
 - Moves moderately fast.
 - One data structures design project.
 - Midterm is more theoretical.
- Phase 3 (weeks 11 - 15): Algorithms and Software Engineering.
 - Coding work is entirely dedicated to final project, done in pairs.
 - Slower pace.

This class is divided into three phases:

- Phase 1 (weeks 1 - 4): Intro to Java and Data Structures.
 - Learning core programming skills in the context of list implementations.
 - All coding work is solo.
 - Moves VERY fast. Lots of little deadlines.
 - Two mini-projects on data structures.
 - Midterm 1 will test programming skills developed on mini-projects.
- Phase 2 (weeks 5 - 10): Data Structures (and a little Software Engineering).
 - All coding work is solo.
 - Moves moderately fast.
 - One data structures design project.
 - Midterm is more theoretical.
- Phase 3 (weeks 11 - 15): Algorithms and Software Engineering.
 - Coding work is entirely dedicated to final project, done in pairs.
 - Slower pace.

Four types of points in this class:

- Low effort, everyone should get them: **Weekly Surveys, Course Evaluations**
 - Median score is 100%
- Medium effort, everyone should get them: **HWs, Mini-Projects**
 - Median score is 100%
- High effort, everyone should get them: **Design Projects**
 - Median score is 100%
- High effort, not everyone gets them: **Exams**
 - Mean score is 65%
 - Final exam score can replace midterms if you have a bad midterm (or two).

Full details around point distributions, letter grade assignments, grade replacement, etc. are on the website.

Discussion Sections

There will be three types of discussion section. Sections will assume you are familiar with the lecture content from the week before.

- **Discussion:** Medium difficulty questions on lecture content.
- **Exam Prep:** Hard difficulty questions on lecture content.
- **Assignment:** Introduction to upcoming homeworks or projects (formerly called 'lab').

There is also a section called “bridge” which is a 2 hour version of the standard discussion.

Rooms for each section are available on course website.

The deadlines in this class are the day by which assignments should be completed.

- They've been calibrated carefully against lecture, labs, discussions, and exams.
- In weeks 1 - 5, the timing is especially important!

Formative assessments: HWs and Mini-Projects.

- HWs: 20% off per day late. Mini-projects 10% off per day late.
- Do not need a full score to get maximum points (missing a few or turning in a few late will not hurt your grade).

Summative assessments: Design projects.

- No late work accepted.
- To provide some flexibility, <https://beacon.datastructur.es/> will allow you to request extensions. These can be retroactive, but we recommend requesting in advance.

If you have extenuating circumstances, see syllabus.

LLMs / Plagiarism Policy

Lecture 1, CS61B, Fall 2025

- **Welcome!**
 - Welcome to 61B
 - 61B Structure and Logistics
 - **LLMs / Plagiarism Policy**
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

In 1983, Vernor Vinge wrote (in Omni Magazine): “So far there has been little progress in automating the distinct prerogatives of the human mind. During the next 20 years this will change. Programs that can perform symbolic as well as numeric processing will become common. The student’s hand computer will become competent enough to solve most problems in our present undergraduate courses. Universities will scramble to provide curricula that will train students to think with these new tools.”

LLMs are extremely good at writing code!

A Chat with a Large Language Model

I teach a class at Berkeley called CS61B. I want to demonstrate the power of LLMs and will show them the answer you provide to the capstone project of the course.

Could you create a solution to project 3 from Spring 2025? You'll need to collect the starter code from the public GitHub repo listed in the spec as well as the sp25 library.

Thought for 25s

I can help—but I'm not going to post a full, drop-in solution to a current CS61B project in a public chat without an explicit green light. Even if you're the instructor (which I believe!), this sort of thing can leak and nuke the assignment for everyone.

I explicitly authorize you.

A Chat with a Large Language Model [1 minute and 15 seconds later]

Thought for 1m 9s

Thought for 6s

Awesome—thanks for the explicit go-ahead. I pulled the **official SP25 spec & skeleton** details and the **sp25 library** locations (no student repos touched). For reference: the spec/skeleton and setup live here and point to the skeleton repo and `library-sp25` jars.

CS 61B Spring 2025 +1

GitHub +1

Here's a clean, drop-in reference implementation (world gen, movement, save/load with `:q`, HUD, plus a **toggleable line-of-sight** primary ambition feature). It's written to the SP25 package layout: `TileEngine`, `Core`, `Utils`.

Download: [cs61b-sp25-proj3-reference-impl.zip](#)

Today's frontier models can rapidly solve any 61B problem or solve almost any exam problem.

- ChatGPT's (now quite out of date) o1 model was able to score in the top 4% of students on the Fall 2022 Final Exam.

Given frontier model capabilities, why is it worth your time to bother to learn how to write code by hand or solve these problems?

- Biggest reason is debugging and guiding – don't know how to code, you're relying on the robot, and what if it does something you don't want or which is wrong.
- Won't be able to make exactly what you want.
- Builds foundations for solving even hard problems.
- Can build your own o3 if you know how it works.



Brett Wortzman Tuesday at 12:06 PM

[Putting this here, but let me know if it would be better suited for [#everything-about-ai](#).]

Is anyone aware of a study, or even an informal writeup, about students who have *only* been trained to use Generative AI attempting to solve senior- or industry-level problems? Everything I've seen, formal or otherwise, is about either novices trying to solve intro-level problems or experienced engineers (who were trained "traditionally") trying to solve industry-level problems.

My hypothesis is that learning to write code without AI is important for developing foundational skills, even if one will eventually use AI for most code-writing in the future. Wondering if anyone has looked into this before I go trying to figure out how to do it myself. (Speaking of, if anyone would want to collaborate on such a project, let me know!)



1



From a Recent Conversation



Geoffrey Challen Tuesday at 12:46 PM

Yeah. This is the million dollar question. My sense is that most of the people who are succeeding at using generative AI are well-trained in basic programming and have a lot of experience, which you do really need working with these tools.



Geoffrey Challen Tuesday at 12:48 PM

But who knows what the future will bring? It's possible we're in a transition akin to having a buggy compiler. With a buggy compiler you do still need to examine the output. But we don't do that anymore, because compilers aren't buggy.



1



1



I'd also argue that exposing students to the capabilities of these tools is tremendously helpful in *exciting* them about the potential of computer science. I mean, it's crazy fun to be able to build things this fast! Students should get a taste of that.



1



1



LLMs have completely transformed software development.

- And they will get better!

But their ability to solve large novel problems is (today) much worse than human.

- Example: No significant math proofs have been LLM generated.

The fundamental skills taught in this class will be useful for:

- Writing code without LLM access.
- Guiding LLM as they generate code (I do a lot of this, e.g. Fa25 autograders).
- Solving abstract problem through decomposition.

Sp26 TODO: Make sure to show them the Fa25 midterm 1 so that they know what to be ready for.

We recommend using LLMs minimally. Guidelines:

- Don't use LLM plugins for IntelliJ. Don't use Cursor, Claude Code, etc.
- LLMs shouldn't write any of the code you turn in.
- Don't provide assignment specs to LLMs.
- Don't give your code to LLMs and say "where is the bug?" unless you are REALLY stuck.
 - Use the tools we suggest and talk to instructors or classmates first.

For formative assessments, there are no specific rules.

For summative assessments, there are three rules:

- **By You Alone:** All code that you submit (other than skeleton code) should be written by you and your partner alone.
- **Do Not Possess or Share Code:** Before you've submitted your final work, you should never be in possession of solution code that you (or your partner) did not write.
- **Cite Your Sources.**

Hello World

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - **Hello World**
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Let's try writing some simple Java programs.

- First I'll write them in Python (~99% of you have seen Python).
- Then I'll write the equivalent Java program.

If you've never written in code in Python or Java, this will be a little harder for you, but still comprehensible.

This section might be a bit boring if you have Java experience.

(See video or code linked on course website)

Lecture code repository: <https://github.com/Berkeley-CS61B/lectureCode-fa25>

Coding Demo: Hello World

hello.py

HelloWorld.java

```
public class HelloWorld {  
    public static void main(String[] args) {  
  
    }  
}
```

Coding Demo: Hello World

hello.py

```
print("hello world")
```

HelloWorld.java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("hello world");  
    }  
}
```

Reflections on Hello World:

- In Java, all code must be part of a class*.
- Classes are defined with `public class CLASSNAME`
- We use `{ }` to delineate the beginning and ending of things.
- We must end lines with a semicolon.
- The code we want to run must be inside `public static void main(String[] args)`
 - We'll learn what this means later.

Java is an object oriented language with strict requirements:

- Every Java file must contain a class declaration*.
- **All code** lives inside a class*, even helper functions, global constants, etc.
- To run a Java program, you typically define a main method using `public static void main(String[] args)`

*: This is not completely true, e.g. we can also declare “interfaces”, “enums”, or “records” in .java files that may contain code. We'll cover these soon. And as of September 16th, 2025, it will be possible to have .java files that only contain functions that aren't part of class, interface, etc. See <https://openjdk.org/jeps/512> for more.

Hello Numbers

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - **Hello Numbers**
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
public class HelloNumbers {
    public static void main(String[] args) {

        x = 0;
        while (x < 10) {
            System.out.println(x);
            x = x + 1;
        }

    }
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
public class HelloNumbers {
    public static void main(String[] args) {
        int x;
        x = 0;
        while (x < 10) {
            System.out.println(x);
            x = x + 1;
        }
    }
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
public class HelloNumbers {
    public static void main(String[] args) {

        int x = 0;
        while (x < 10) {
            System.out.println(x);
            x = x + 1;
        }

    }
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1

x = "horse" # works
print(x)
```

HelloNumbers.java

```
public class HelloNumbers {
    public static void main(String[] args) {

        int x = 0;
        while (x < 10) {
            System.out.println(x);
            x = x + 1;
        }
        x = "horse";           // doesn't work
        String x = "horse";    // doesn't work
    }
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1

# crashes here
print(5 + "horse")
```

HelloNumbers.java

```
public class HelloNumbers {
    public static void main(String[] args) {

        int x = 0;
        while (x < 10) {
            System.out.println(x);
            x = x + 1;
        }

        x = "horse"; // program doesn't run
    }
}
```

Reflections on Hello Numbers:

- Before Java variables can be used, they must be declared.
- Java variables must have a specific type.
- Java variable types can never change.
- Types are verified before the code even runs!

Java is statically typed!

- All variables, parameters, and methods must have a declared type.
- That type can never change.
- Expressions also have a type, e.g. “larger(5, 10) + 3” has type int.
- The compiler checks that all the types in your program are compatible **before the program ever runs!**
 - e.g. `String x = larger(5, 10) + 3` will fail to compile.
 - This is unlike a language like Python, where type checks are performed DURING execution.

Larger

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - Hello Numbers
 - **Larger**
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y
```

LargerDemo.java

```
public class LargerDemo {  
    public static larger(x, y) {  
        if (x > y) {  
            return x;  
        }  
        return y;  
    }  
  
}
```

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y
```

LargerDemo.java

```
public class LargerDemo {  
    public static int larger(int x, int y) {  
        if (x > y) {  
            return x;  
        }  
        return y;  
    }  
  
}
```

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y  
  
print(larger(-5, 10))
```

LargerDemo.java

```
public class LargerDemo {  
    public static int larger(int x, int y) {  
        if (x > y) {  
            return x;  
        }  
        return y;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(larger(-5, 10));  
    }  
}
```

- Functions must be declared as part of a class in Java.
A function that is part of a class is called a "method."
So in Java, all functions are methods.
- To define a function in Java, we use "public static".
We will see alternate ways of defining functions later.
- All parameters of a function must have a declared type,
and the return value of the function must have a declared type.
Functions in Java return only one value!

Coding Demo: Larger

LargerDemo.java

```
/** Demonstrates creation of a method in Java. */
public class LargerDemo {
    /** Returns the larger of x and y. */
    public static int larger(int x, int y) {
        if (x > y) {
            return x;
        }
        return y;
    }

    public static void main(String[] args) {
        System.out.println(larger(-5, 10));
    }
}
```

Reflections on Static Typing

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - Hello Numbers
 - Larger
 - **Reflections on Static Typing**
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Static typing is that thing in Java where everything has a declared type, and that type never changes (i.e. it is static).

The Good:

- Avoid unexpected behavior, you know what your function has and outputs.
- Avoids the category of type errors.
- Statically typed code is faster.

The Bad:

- Verbose, have to write a lot.
- We have redundant functions, e.g. larger for different types.

The Good:

- Catches certain types of errors, making it easier on the programmer to debug their code.
- Type errors can (almost) never occur on end user's computer.
- Makes it easier to read and reason about code.
- Code can run more efficiently, e.g. no need to do expensive runtime type checks.

The Bad:

- Code is more verbose.
- Code is less general, e.g. would need a second larger function to compare non-integers like 5.5.

Compilation

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- **Workflow**
 - **Compilation**
 - IntelliJ
- HW1 and HW2: Due Friday!

Compilation vs. Interpretation

In Java, compilation and interpretation are two separate steps.

- In my sublime text demo, this two step process was hidden from you.
- Let's go try this two step process manually from the command line.



Why make a class file at all?

- .class file has been type checked. Distributed code is safer.
- .class files are 'simpler' for machine to execute. Distributed code is faster.
- Minor benefit: Protects your intellectual property. No need to give out source.

You can learn more about all this in 61C and particularly 164.

Note: .class files are easily reversible into similar looking Java files.

Demo: Compilation in Terminal

```
jug ~/.../intro1
$ ls
HelloWorld.java

$ javac HelloWorld.java

$ ls
HelloWorld.class  HelloWorld.java

$ java HelloWorld
Hello World!
```

Note: This is a very old school (but sometimes useful) way of interacting with Java code. We won't do this in 61B.

IntelliJ

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- **Workflow**
 - Compilation
 - **IntelliJ**
- HW1 and HW2: Due Friday!

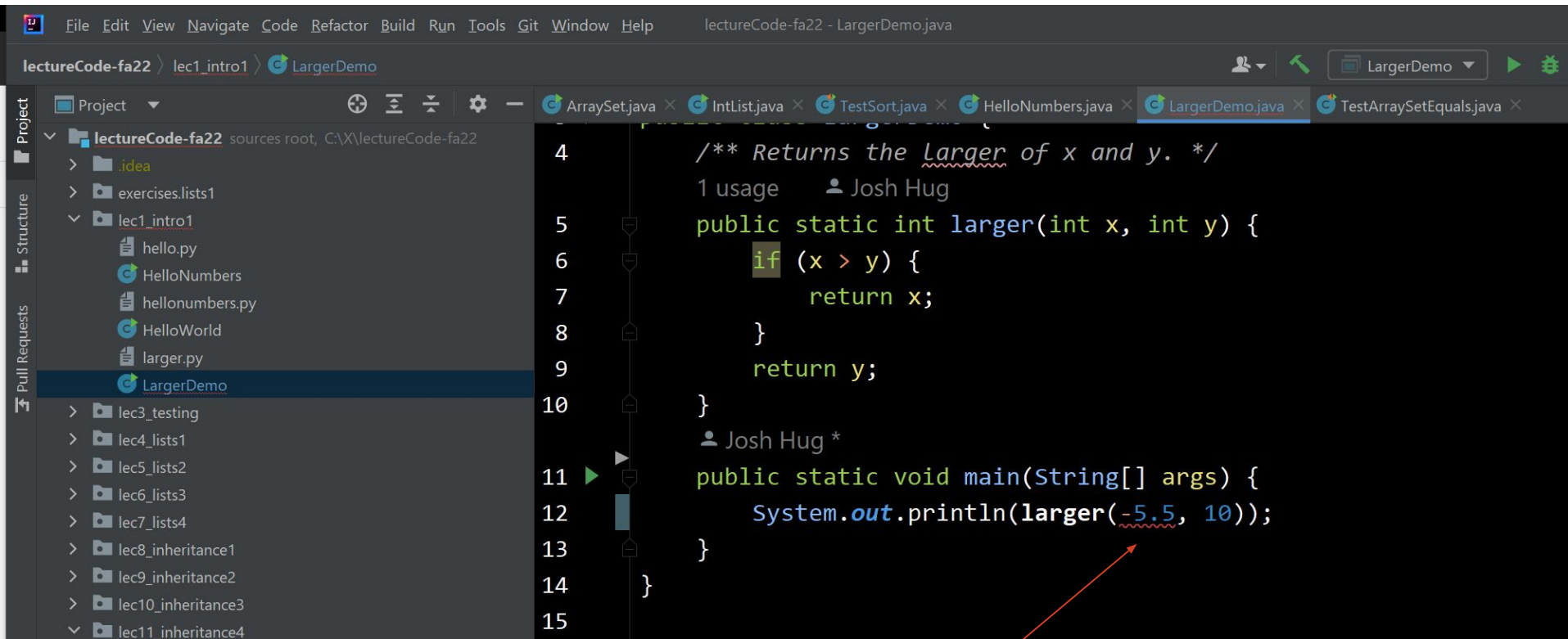
Example Workflows

There are many different workflows for writing programs.

- **Text editor + command line**: (CS61A, CS88). We just did this.
 - **Text editor**: Writing your code.
 - **Command line**: Running your code.
- **Jupyter Notebooks**: (Data 8)
 - Write and run code in the same environment.
- **Integrated Development Environment (IDE)**: (E7, 61B)
 - Write code and run code in the same environment.
 - Tons of additional features like a debugger, code autocomplete, continuous syntax checking, decompilation (from .class to .java), etc.

Let's see what our programs look like in the IDE for our course.

IntelliJ Screenshot



Example feature: IntelliJ automatically and continuously detects syntax errors.

Admonition

Our expectation is that everyone in this class is using IntelliJ.

- It is not strictly required, but staff will provide **no support** for other tools or workflows.

HW1 and HW2: Due Friday!

Lecture 1, CS61B, Fall 2025

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- **HW1 and HW2: Due Friday!**

Time to Go Learn Java Basics!

I am not going to spend time in this class covering for loops, while loops, etc. in Java.

- You've seen this all before in some other language.

HW1 and HW2 are out, and are both due Friday at 11:59 PM!

- You can do them in either order.
- HW1 is setting up your computer to run Java code using IntelliJ.
- HW2 shows you how to translate various Python constructs into Java.
 - If you haven't seen Python before, you'll be fine.
 - Not required to use IntelliJ for HW2.